Lefèvre's lower bound, and its proof in Rocq

A guide to `Alg2.v`, `Alg1.v` and `Config.v` for readers who do not read Rocq

1 What is being computed

The hard-to-round search needs, for a line $y = ax - b$ over an interval of N points, a **lower bound** on

$$\inf\{b - ax \bmod 1 : x < N\}.$$

If that bound exceeds a threshold, no point of the interval can be a hard-to-round case and the interval is discarded without further work. Lefèvre's algorithm computes such a bound in $O(\log)$ steps rather than $O(N)$.

Everything is scaled by a modulus M so that the development is over natural numbers: $a = A/M$, $b = B/M$. Two functions carry the geometry:

- `pt x` $= (Ax) \bmod M$, the x-th point of the orbit;
- `dst x` $= (B + M -$ `pt x`$) \bmod M$, its distance **down** to b;

and `inf_dst M A B n`, written $\mathrm{Inf}(n)$ below, is the minimum of `dst x` over $x < n$. The theorem to prove is that the algorithm's result is at most $\mathrm{Inf}(N)$.

2 The structure the algorithm walks

2.1 Two-length configurations

Take the first n points of the orbit and cut the circle at them. The **three-distance theorem** says at most three gap lengths occur, and for the values of n the algorithm visits, exactly two: p and q. The state carries those two lengths together with how many gaps of each there are, u and v. This is the record `inv`, whose essential clauses are

```
u * p + v * q = M     p = pt v     q = M - pt u
```

The first says the gaps tile the circle; the other two say the two lengths are themselves points of the orbit. A second record, `invx`, fixes the **index** structure — which point succeeds which:

```
z < u ⟹ pt (z + v) = pt z + p     u <= z ⟹ pt (z - u) = (pt z + q) mod M
```

So an index below u heads a gap of length p, and an index at least u heads one of length q. That single fact is what lets the proofs say "b is in a p-gap" without any geometry: it means the index of the point just below b is smaller than u.

2.2 One step

Passing from n points to more points is a Euclidean reduction on the pair (p, q). Reducing the larger gap q by k copies of p splits each q-gap and updates the counts:

```
q -= k*p, u += k*v        or        p -= k*q, v += k*u
```

The paper's Property 3 says these splits are **directional**: a q-gap becomes k gaps of length p followed by the residual, left to right, whereas a p-gap becomes the residual followed by k gaps of length q — points enter from the right. That asymmetry is visible throughout the proofs.

`Config.v` proves what one reduction does to a configuration, **for any k up to the quotient**: it preserves `inv` (`inv_red_lt`, `inv_red_ge`) and `invx` (the `invx_red_*` families), and it moves the infimum by a known amount. Algorithm 2 always takes k maximal; Algorithm 1 takes it maximal on one side of a turn and $k = 1$ on the other, so both are instances of the same lemmas.

3 Algorithm 2

3.1 The loop

One turn is one reduction, chosen by comparing the two lengths, together with an update of a recorded distance d:

$$\texttt{step p q d u v} = \text{if } p < q \text{ then reduce } q \text{ else reduce } p$$

and `run` iterates it until $u + v$ reaches N, returning d.

3.2 Why the result is a lower bound

The interesting variable is d. It is **not** the infimum: batching the reduction makes the loop overshoot, so d can already refer to a point beyond the current count. What is maintained instead is the record `invd`, three clauses:

```
d < maxn p q     d <= Inf (u + v)     d = Inf (u + v) mod p
```

— d is below the infimum, and congruent to it modulo the smaller gap. The second clause is what makes the returned value a lower bound; the third is what lets it survive a reduction, because a reduction changes the infimum by a multiple of p.

The main work is showing the three records survive a step. For `inv` and `invx` this is the tiling bookkeeping. For `invd` it is a statement about where the new points land relative to b, and it splits by which gap is being reduced:

- reducing q (`inf_new_eq_lt`): every point walks down by steps of p, so the new infimum is exactly $\mathrm{Inf} \bmod p$ — clean, because the walk can be iterated until it can go no further;
- reducing p (`ge_inf_alt`): points enter p-gaps from the right, so whether the infimum moves depends on which gap holds b. The result is a **disjunction**: either the new infimum is the new d, or it is unchanged and was already below q.

That asymmetry is not an artefact of the formalisation; it is Property 3.

Two lemmas do the geometric work and are reused constantly: `gap_p_empty` and `gap_q_empty` say that a point with b inside its own gap is the nearest point below b — nothing else in range can be closer. A third, `gap_walk`, is the tiling in index form: for any two indices in range,

```
dst y - dst z = a*p + b*q   with   a*v + y = b*u + z,
```

with $a \le u$ and $b \le v$. Most "no point can be there" arguments are one application of it.

The soundness theorem is `lefevre_sound`, and the form the search uses is `lefevre_test`: if the returned bound clears ε, then every $x < N$ has `dst x` above ε.

4 Algorithm 1

4.1 Why it is not just Algorithm 2 again

Algorithm 1, the original, differs in two ways that matter to the proof.

First, **one turn is two reductions**: a division step and then a single subtraction. Second — and this is the awkward part — **the exit test sits between them**. The loop tests the count after the division half, and if it does not exit, performs the subtraction half, which adds more points without testing again. Lefèvre says so explicitly: the algorithm stops not at N but at the first configuration size at least N.

So the file has `half1` and `half2`, and `half1` is `Alg2.step` while `half2` is the same reduction at $k = 1$.

4.2 What d is, and where

Both the paper and the thesis say d is the distance from b to the nearest point on its left. That is true — **between the halves**, which is where the exit test reads d and where the loop returns it. It is not true at the top of a turn, because `half2` has just added points and left d untouched. Algorithm 2′s `invd` does not hold here at all; it already fails at the initial state.

The invariant that does hold at a turn start is the record `invw`:

$$\texttt{d < p + q} \qquad \mathrm{Inf}(u+v) = \texttt{if d < p then d else d - p}$$

In words: d is the infimum plus the one p-step that `half1` has not yet taken — and which of the two cases holds is exactly what the branch test decides. Consequently `half1_exact` gives $d = \mathrm{Inf}$ after the division half, and the returned value is an infimum over a range at least as large as N, which is the bound.

4.3 The third clause

`invw` has a third field, and it is the one place where the formalisation had to add something the sources only assert. §4.1 remarks that

"the condition at line 4 is true if b is in an interval of length p"

which is what makes the six-case analysis work. One direction is immediate: if b is in a p-gap then $d < p$, since d is a distance inside that gap. The converse is **not** a property of the state — when the infimum is below both gap lengths, the configuration alone does not say which gap holds b (Alg2′s `ge_inf_le` gives only the two one-way implications). It is a property of the states the loop reaches, kept true by Property 3′s directionality. So it is carried as an invariant clause,

$$\texttt{(y < u) = (d < p)} \qquad \text{for the index } y \text{ attaining the infimum,}$$

using `invx_p1`/`invx_p2` to read "index below u" as "p-gap".

4.4 The turn, case by case

With that in place the two halves are:

- **`half1`** does not move the infimum, in either branch. If $d < p$ then d is already the infimum, so the reduction of q cannot lower it (`half1_inf_lt`). If $p \le d$ then b sits in a q-gap, and reducing p splits p-gaps only, so no point enters below b (`half1_ge_nodrop`). What it does is subtract the pending p from d.
- **`half2`** adds points and leaves d alone, so it is where the infimum drops — by nothing, or by the new p — and where the invariant is restored (`invw_sub_p`, `invw_sub_q`). The two sides are not symmetric, again by Property 3: on the q side the two cases agree with the test on their own, while on the p side the lemma must be told that b is in a p-gap.

4.5 Soundness

`run1_sound` is an induction on the fuel. Each turn: `half1_exact` makes d the infimum, the exit case is `half1_leq_inf`, and the recursive case rebuilds the three records. The overshoot — a turn beginning with the count already past N — is closed by `run1_past`, which needs only `invw`: the loop returns at most the d that `half1` leaves, and that is an infimum over a range at least as large as N.

The results are

```
lefevre1_sound : 2 < N -> lefevre1 M A B N <= Inf N
```

and the corollary the search uses, `lefevre1_test`.

5 State of the development

File	Statements	Contents
`Dist.v`	—	`pt`, `dst`, `Inf` and their arithmetic
`Alg2.v`	99	Algorithm 2: `step`, `run`, `inv`/`invd`/`invx`, the gap and walk lemmas, `lefevre_sound`
`Config.v`	25	one reduction at an arbitrary k: `inv` and `invx` preserved, and how far the infimum drops
`Alg1.v`	52	Algorithm 1: `half1`/`half2`, `run1`, `invw`, `lefevre1_sound`

Both soundness theorems, and both `_test` corollaries, are proved without axioms: `Print Assumptions` reports **closed under the global context** for each.

Neither algorithm is exact — both can return a bound strictly below the true infimum, and the files record the smallest witnesses. Algorithm 1 is the sharper of the two; that comparison is measured but not proved.

Two things remain open in the organisation rather than the mathematics. `Config.v` still imports `Alg2.v`, so Algorithm 2′s step lemmas are independent proofs of what Config proves at general k rather than instances of it; folding them together would remove that duplication. And several range hypotheses in `Alg2.v` are of the form "the count is below N" where "below the orbit length $M/\gcd(A, M)$" would do, which is what `Config.v` now uses.